



Inspiring Excellence

CSE422 - Artificial Intelligence

Software Quality Classification

ALENA HALDER TAMO

22301046

Table of Contents

1. [Introduction](#)
2. [Dataset Description](#)
 - [Imbalanced Dataset](#)
 - [Exploratory Data Analysis](#)
3. [Dataset Pre-processing](#)
4. [Dataset Splitting](#)
5. [Model Training & Testing](#)
6. [Model Comparison](#)
7. [Conclusion](#)

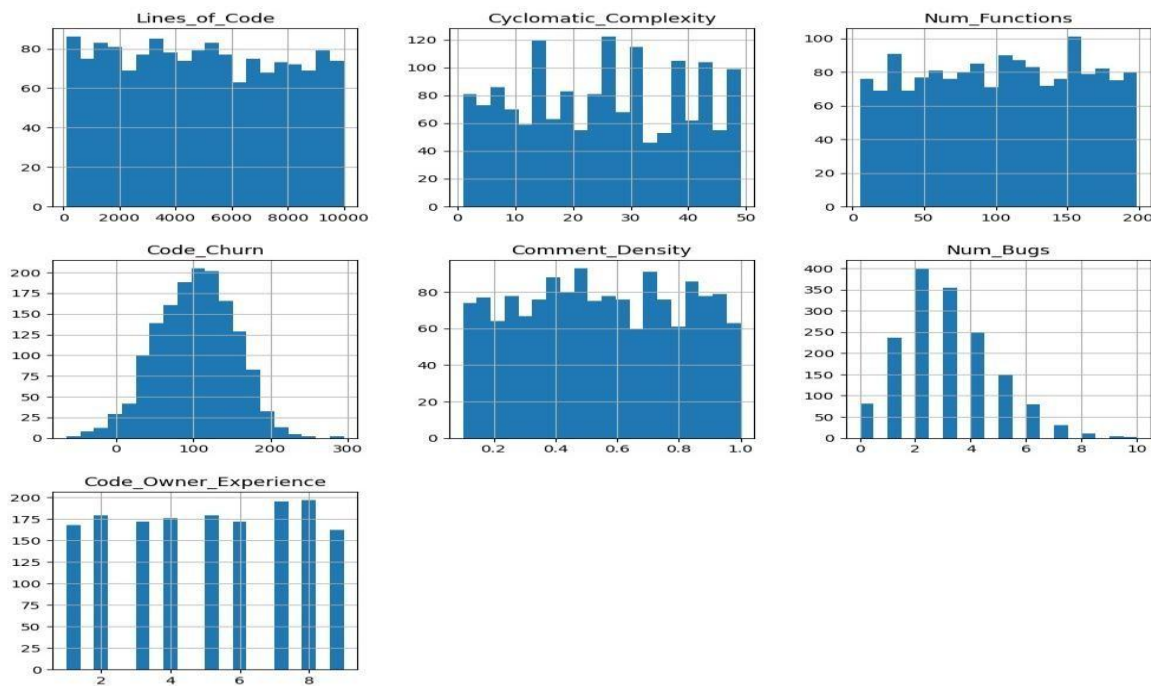
1. Introduction

This project aims to develop a machine learning model to predict the quality of software components based on static code metrics such as lines of code, cyclomatic complexity, code churn, and more. The objective is to support early-stage software quality assurance efforts using machine learning models. By analyzing these metrics, we attempt to classify software quality into three categories: Low, Medium, and High.

The motivation behind developing this model to predict software quality lies in improving efficiency and reducing costs. Detecting defects early allows developers to focus on high-risk areas, optimizing resources, and reducing the time spent on manual testing. A data-driven approach enhances decision making, making software maintenance more proactive and effective. It also automates parts of the quality assurance process, increasing productivity while minimizing the risk of human error. Ultimately, such a model ensures better software quality, faster development cycles, and more reliable end products. By leveraging machine learning techniques, we aim to create a scalable and reliable model capable of analyzing software quality, contributing to better outcomes.

2. Dataset Description

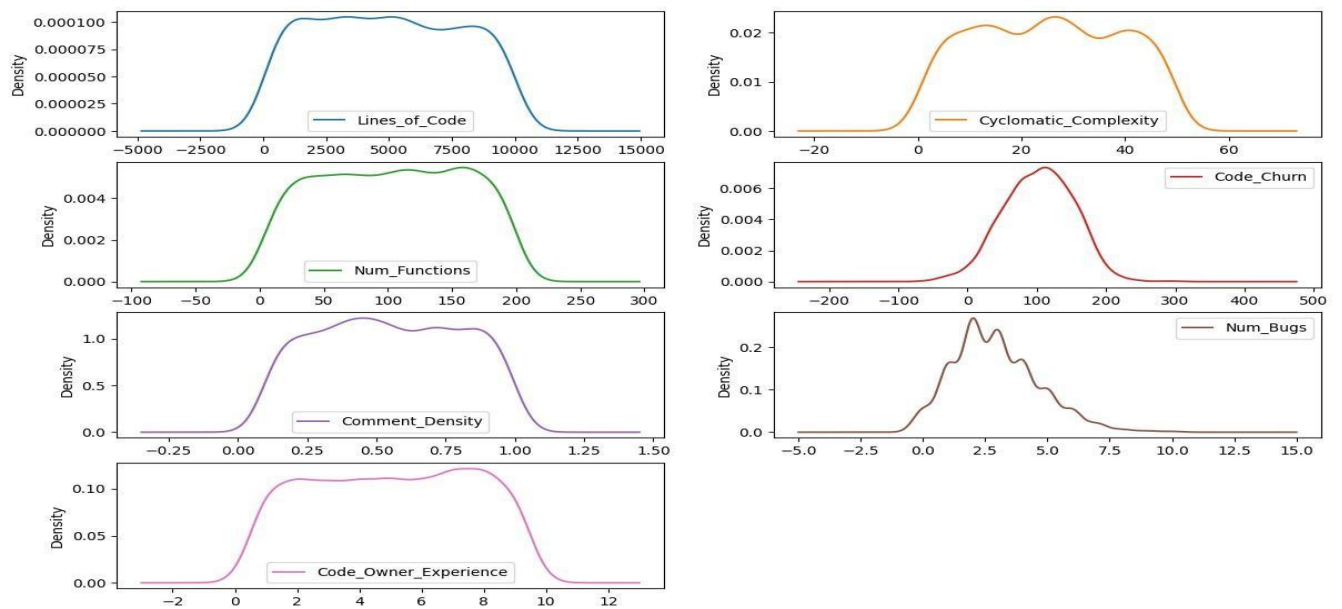
- Features: 9 (8 input features, 1 output feature)
- Total Data Points: 1600
- Target Variable: **Quality_Label** (Low, Medium, High)
- Problem Type: This is a **multiclass Classification** problem that aims to predict whether the software quality label is low, medium, or high.

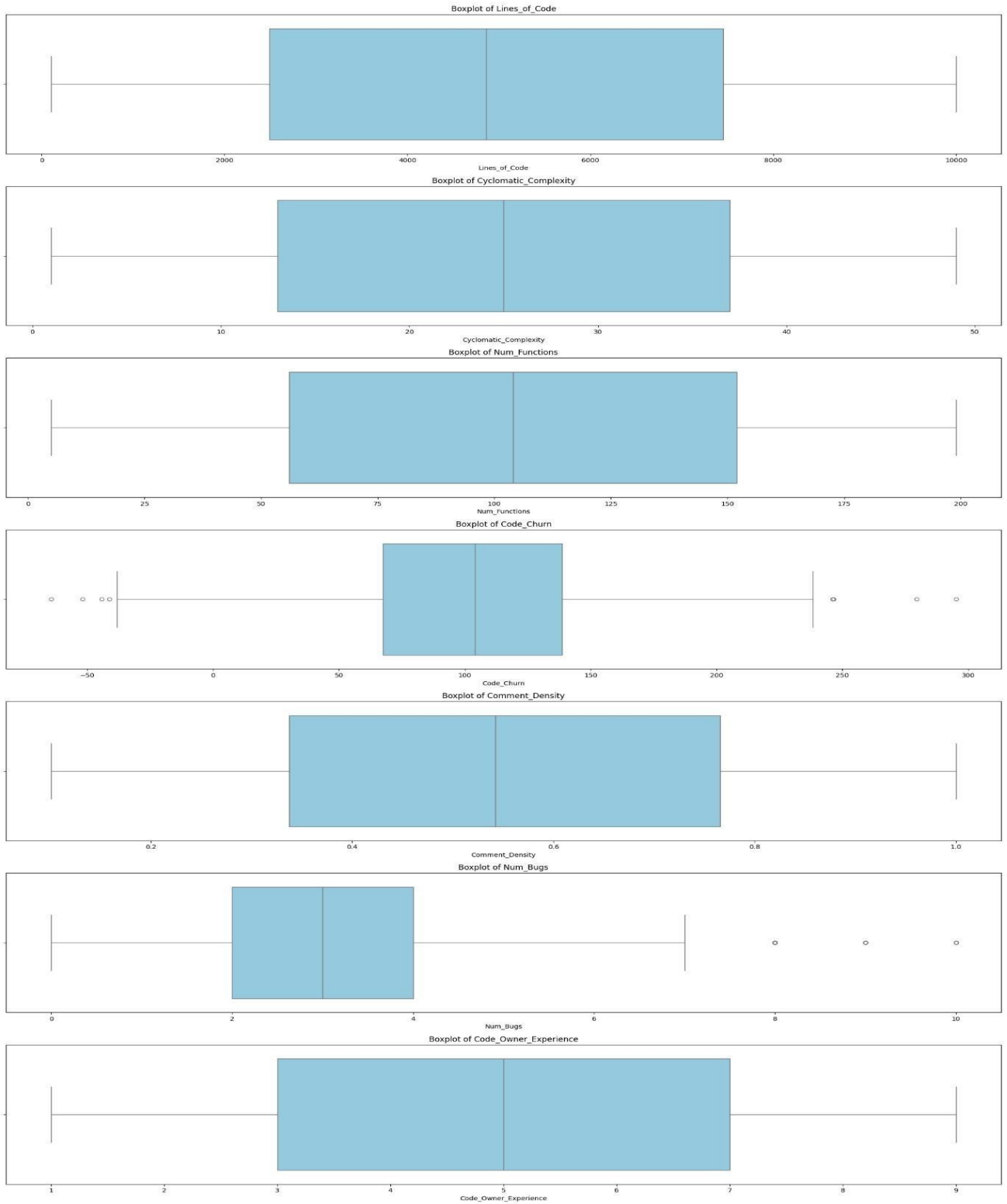


Feature Types:

- Quantitative:
 - Lines_of_Code
 - Cyclomatic_Complexity
 - Num_Functions
 - Code_Churn
 - Comment_Density
 - Num_Bugs
 - Code_Owner_Experience

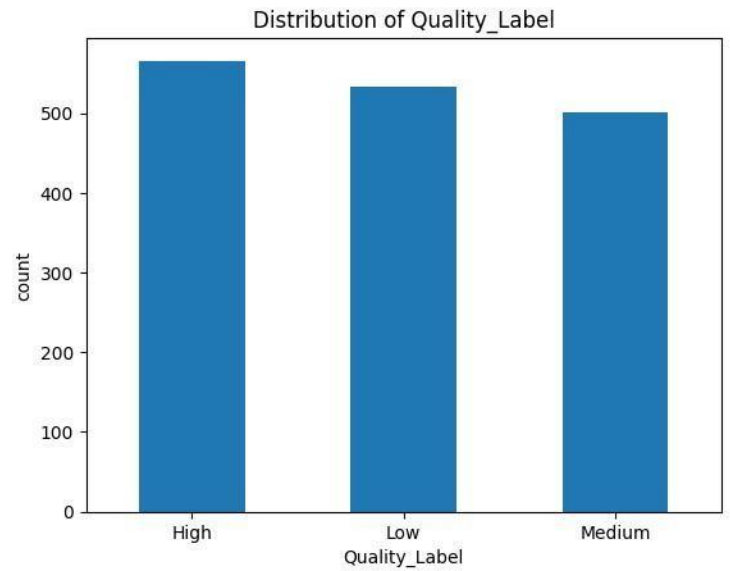
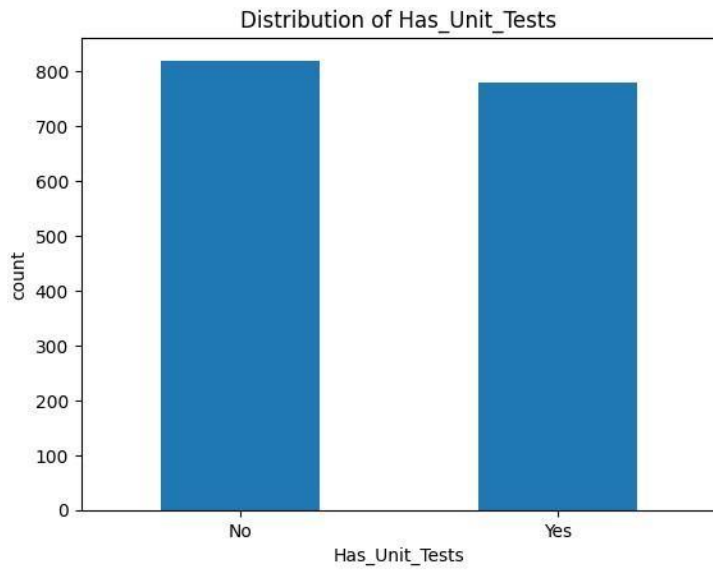
Density plot of Numerical features





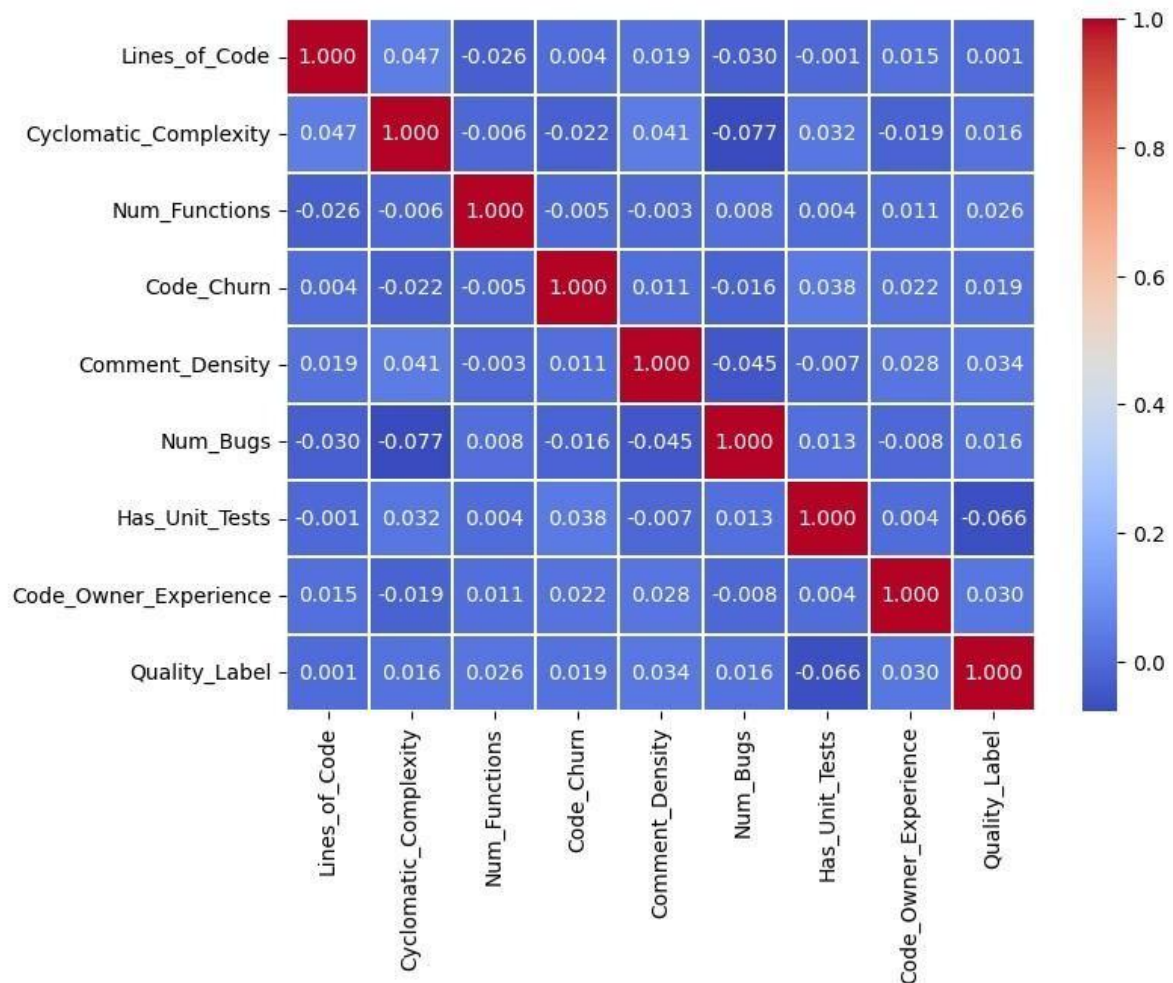
- Categorical:

- Has_Unit_Tests (Yes/No)
- Quality_Label (High/Medium/Low)



Correlation Between All Features:

To understand feature relationships and their relevance to the target variable (quality label), a correlation matrix was created using libraries like `Pandas`, `Seaborn`, and `Matplotlib`. From the heatmap, we understand that the warmer relation means the features are strongly related, while the cooler it means they are least correlated. The features are most strongly correlated with themselves, but we see the correlation with other variables is very low, some even going into negative correlation.



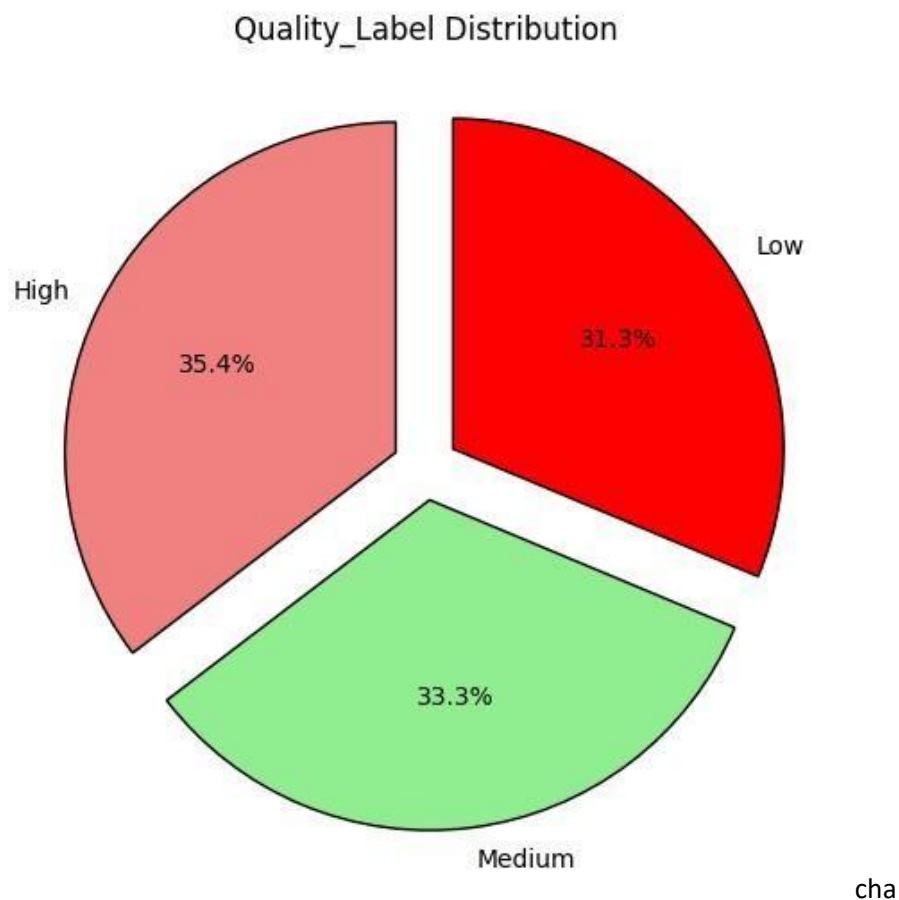
As our target feature is `Quality_Label`, we look at the correlation of this feature with the rest of the features. Although the values are low, we see all the features except `Has_Unit_Tests` are positively correlated, while the mentioned feature is negatively correlated.

Balanced/Imbalanced Dataset:

The target feature (Quality_Label) has three classes:

- High
- Medium
- Low

The **Quality_Label** class distribution was visualized using a Pie Chart. While the classes were not severely imbalanced, there were differences in the number of samples per class that could affect classifier performance. 35.4% of the data was marked High, 31.3% was marked Low, and finally 33.3% of the data was marked Medium.



Exploratory Data Analysis

In our Exploratory Data Analysis (EDA), we employed various methods to better understand the dataset. These methods included distribution tables, density graphs, and histograms. Below is a detailed breakdown of the insights obtained from the analysis:

Box Plot Analysis of Numerical Data

We applied box plots to visualize the distribution and detect any potential outliers in the numerical data:

1. Lines_of_Code:

- The distribution of **Lines_of_Code** appears symmetrical, with the median line positioned well in the center. The whiskers extend over a reasonable range, and a few data points lie outside the whiskers, indicating the presence of potential outliers.

2. Cyclomatic_Complexity:

- The **Cyclomatic_Complexity** boxplot shows a consistent distribution. The median is located near the center, and the whiskers extend adequately in both directions. The distribution does not show any significant skew, and the outliers are not particularly pronounced.

3. Num_Functions:

- The distribution of **Num_Functions** is relatively balanced, with the median dividing the data evenly. A few outliers are observed at higher function counts, suggesting that some projects have a larger number of functions than the typical range.

4. Code_Churn:

- The **Code_Churn** boxplot reveals significant outliers, with values extending far beyond the typical range. These extreme values may represent exceptional cases where substantial amounts of code are being modified.

5. Comment_Density:

- **Comment_Density** shows a tight distribution, with most values clustered between 0.2 and 0.8. This suggests that the majority of projects in the dataset maintain a fairly consistent level of comment density, though there are some outliers.

6. Num_Bugs:

- The boxplot for **Num_Bugs** displays a relatively tight distribution, with a few outliers. This indicates that most projects have a small number of bugs, though some projects have a larger bug count.

7. Code_Owner_Experience:

- The experience of code owners ranges from 1 to 8 years. Most projects have code owners with 2-4 years of experience, while the whiskers suggest a few cases with higher experience levels.

Key Insights from the Boxplot Analysis

- Outliers: Features like **Code_Churn**, **Num_Bugs**, and **Cyclomatic_Complexity** show notable outliers. These might warrant further investigation, as they could represent exceptional cases or errors in data entry.
- Skewness: While some features, like **Num_Functions** and **Comment_Density**, show a balanced distribution, others like **Code_Churn** are more spread out with extreme cases. This could suggest that certain features have more variability than others.
- Central Tendency: The median line inside each box represents the central tendency of the data, showing where most of the data points are concentrated.

Distribution Analysis of Categorical Data

We also analyzed the distribution of categorical data using histograms:

1. Unit Test Coverage:

- The distribution of unit test coverage suggests a balanced proportion of projects that either have unit tests (**Yes**) or do not (**No**). While the dataset shows a fairly equal split, it is important to note that unit tests are not overwhelmingly skewed in one direction.

2. Quality Label Distribution:

- The **Quality_Label** is fairly balanced across the three categories (**High**, **Medium**, and **Low**), with no category being dominant. This indicates a relatively even distribution of quality labels across the dataset.

Relationship Between **Has_Unit_Tests** and **Quality_Label**

We analyzed the relationship between the presence of unit tests (**Has_Unit_Tests**) and the quality label (**Quality_Label**):

- The grouped bar plot suggests that the presence or absence of unit tests (**Yes** vs **No**) does not drastically influence the distribution of quality labels. Both **High** and **Medium** quality labels are fairly evenly distributed across projects with and without unit tests.

This indicates that unit tests may not have a strong direct association with the quality of the code, as the quality labels remain balanced across both groups.

Correlation Heatmap Matrix

Finally, we examined the correlation between features using a correlation heatmap matrix.

Here are the key observations:

1. Diagonal Values (1.0):

- The diagonal values represent the correlation of each feature with itself, which is always 1.0.

2. Positive Correlations:

- Cyclomatic_Complexity and Num_Functions show a small positive correlation (0.047). This suggests that as cyclomatic complexity increases, the number of functions in a project also increases slightly.
- Code_Churn and Comment_Density have a small positive correlation (0.038), indicating that projects with more churn tend to have more comments.

3. Negative Correlations:

- Lines of Code and Num_Bugs show a small negative correlation (-0.030), suggesting that projects with more lines of code might have fewer bugs, although this relationship is weak.

- Has_Unit_Tests and Code_Churn show a small negative correlation (-0.045), suggesting that projects with more churn might have fewer unit tests.

4. Weak Correlations with **Quality_Label**:

- The correlation between the target variable **Quality_Label** and other features is generally weak. For instance, the correlation with Code_Owner_Experience is 0.030, and with Num_Bugs is 0.016. This suggests that the features do not have a strong linear relationship with the quality label, which could mean that other factors or more complex relationships may be at play.

3. Dataset Pre-processing

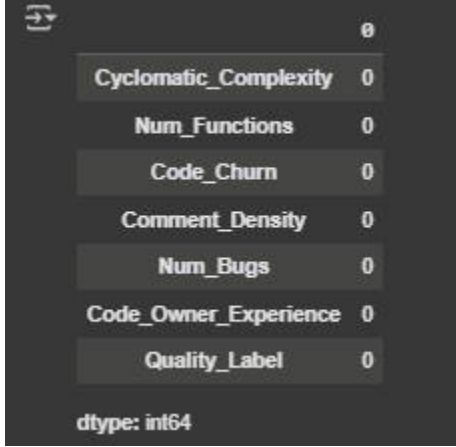
The following pre-processing techniques were applied:

- **Handling Missing Values:** The 'Code_Churn' and 'Comment_Density' columns contained missing values. These were filled using the mean of the available values in the columns 'Code_Churn' and 'Comment_Density'. This common imputation method maintains the overall distribution of the data.



	0
Cyclomatic_Complexity	0
Num_Functions	0
Code_Churn	80
Comment_Density	80
Num_Bugs	0
Code_Owner_Experience	0
Quality_Label	0
dtype: int64	

=====>



	0
Cyclomatic_Complexity	0
Num_Functions	0
Code_Churn	0
Comment_Density	0
Num_Bugs	0
Code_Owner_Experience	0
Quality_Label	0
dtype: int64	

- **Categorical Encoding:** The categorical feature “Has_unit_Tests” was converted into a numerical format for compatibility with machine learning models. This encoding method allows algorithms to interpret the qualitative variables without assuming any ordinal relationships.

- Has_Unit_Tests → Yes = 1, No = 0

- **Label Encoding:**

- Quality_Label was converted to numerical form using label encoding.

- **Feature Scaling – Normalization:** Scaling makes sure all features have the same importance by bringing them to a similar range. It helps models like neural networks learn faster and more effectively. For models like KNN, it prevents one feature from overpowering others in calculations. For example, before scaling the variances of cyclomatic_complexity, num_functions, and code_churn were much higher than the other features, and after scaling, the new variance of all features seems more balanced than the previous one.

terminal	ipython
variance of all features:	variance of all features:
Cyclomatic_Complexity 192.643868	Cyclomatic_Complexity 0.083613
Num_Functions 3088.661810	Num_Functions 0.081854
Code_Churn 2427.104879	Code_Churn 0.018788
Comment_Density 0.062324	Comment_Density 0.077124
Num_Bugs 2.944409	Num_Bugs 0.029444
Code_Owner_Experience 6.552070	Code_Owner_Experience 0.102376
Quality_Label 0.665641	Quality_Label 0.166410
dtype: float64	dtype: float64

4. Dataset Splitting

In this section, we performed the train-test split to prepare the data for model training and evaluation. The dataset was split into training and testing subsets, with 70% of the data used for training and 30% reserved for testing.

The features (X) were derived by excluding the target column, Quality_Label, which is our dependent variable. The target variable (y) was set to the Quality_Label, which contains the class labels for each project.

After splitting the data, we obtained the following:

- Training Data:

- X_train: 1120 samples and 6 features.
- y_train: 1120 target labels corresponding to the training samples.
- Testing Data:
 - X_test: 480 samples and 6 features.
 - y_test: 480 target labels corresponding to the test samples.

Next, the target labels (`y_train` and `y_test`) were encoded using `LabelEncoder` to transform the categorical labels (`High`, `Medium`, `Low`) into numerical values (0, 1, 2). The encoded labels were used for model training and evaluation, while the features (`X_train` and `X_test`) remained unchanged.

The dataset has been successfully split into training and testing sets, with the training data containing 1120 samples and the testing data containing 480 samples.

5. Model Training and Testing

In this section, we trained and evaluated three different classification models:

1. K-Nearest Neighbors (KNN):

- The KNN classifier was trained using the default number of neighbors (5). After training, predictions were made on the test set, and the model's performance was evaluated.

2. Naive Bayes:

- We applied the Naive Bayes classifier to the original training data and evaluated its performance on the test set. The training process involved fitting the Naive Bayes model, followed by predictions on the test set.

3. Neural Network (MLPClassifier):

- For the neural network, we utilized a multi-layer perceptron (MLP) model. The network was trained using a `Sequential` model with two hidden layers and dropout for

regularization. After training, predictions were made, and the performance was assessed.

Each model was evaluated based on several metrics, including accuracy, precision, recall, and F1-score.

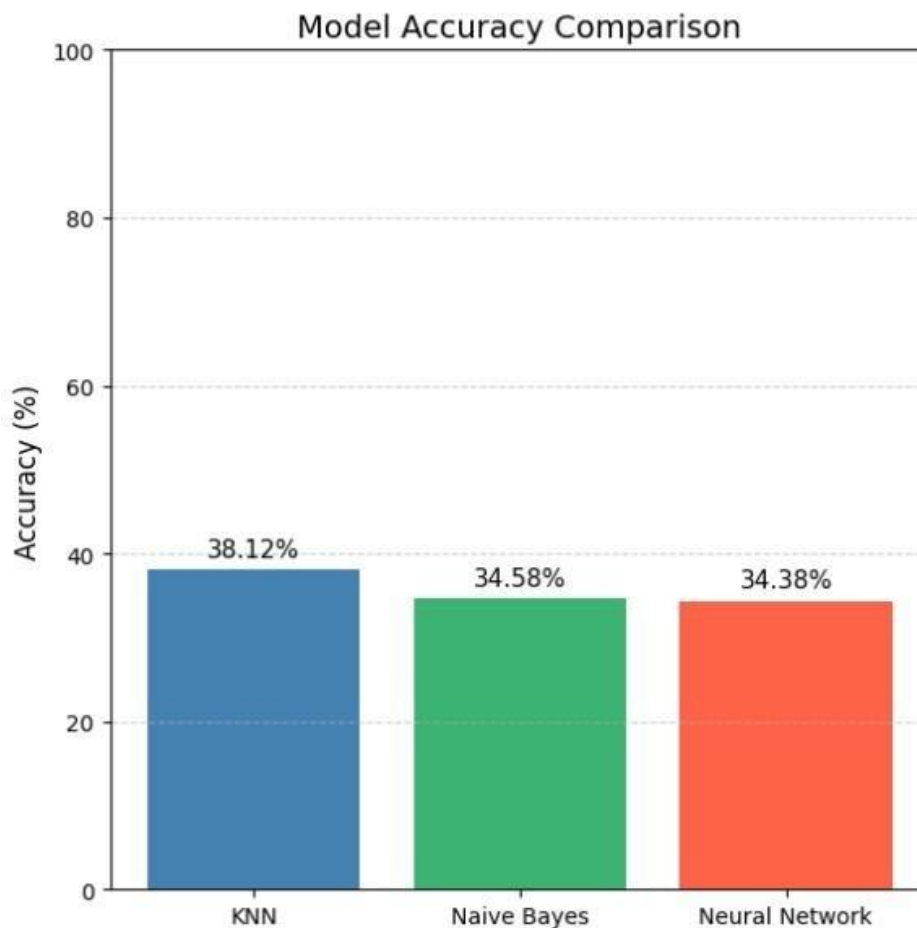
The results from the classification reports are summarized in the following tables.

6. Model Selection and Comparison

After training the models, we compared their performance using several evaluation metrics:

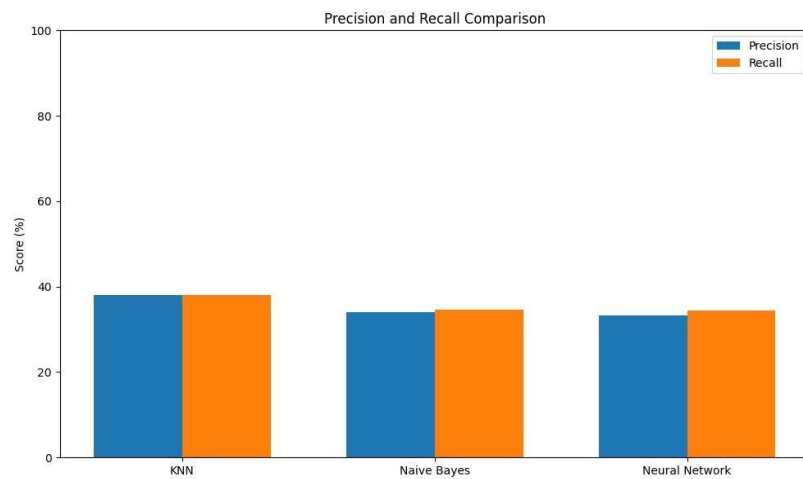
- **Model Accuracy Comparison:**

The bar chart visualizes the accuracy for each model. The K-Nearest Neighbors (KNN) classifier achieved an accuracy of 38.12%, outperforming both Naive Bayes at 34.58% and the Neural Network (MLP) at 34.38%.



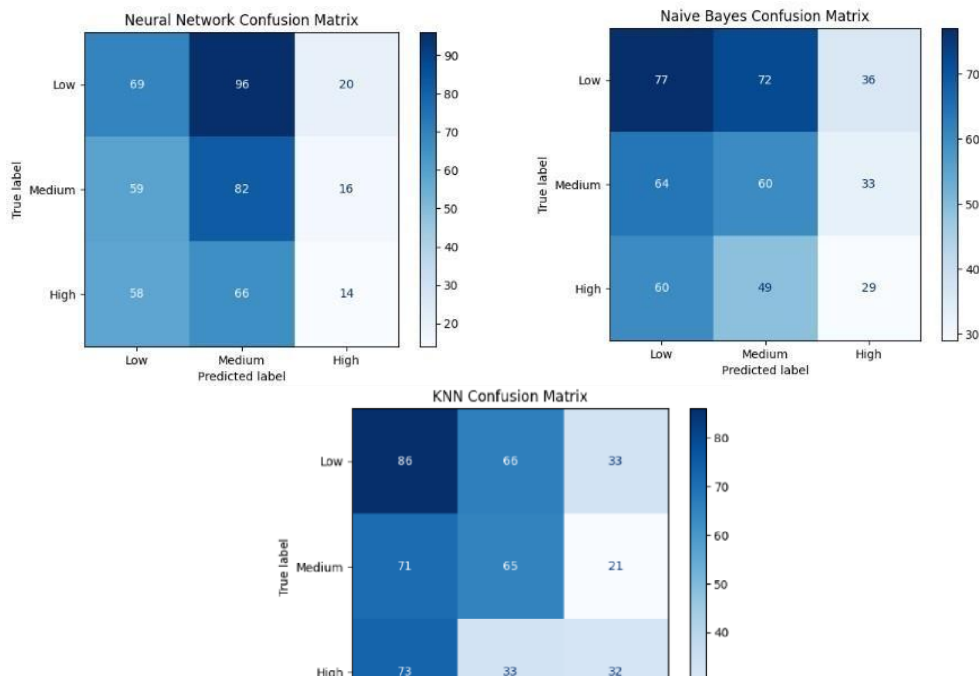
- **Precision and Recall:**

The precision and recall comparison highlights the performance of each model for each class. The KNN model showed slightly better precision and recall for most classes compared to Naive Bayes and Neural Network, but the differences were still marginal.



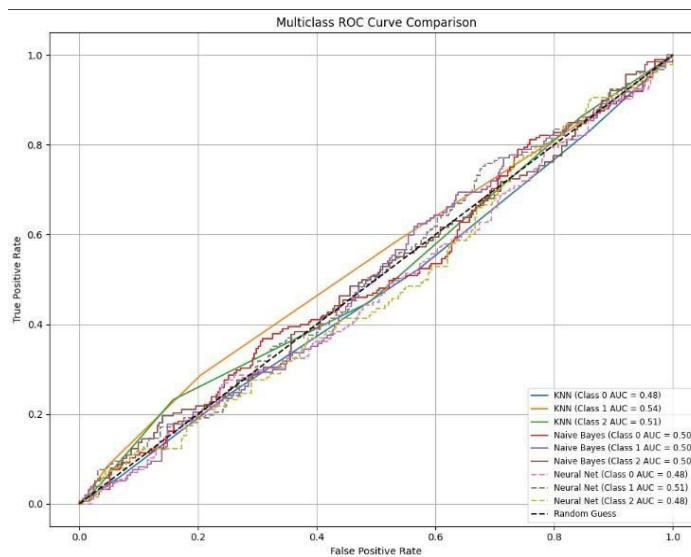
- **Confusion Matrix:**

The confusion matrices for each model provide a more detailed view of how the models performed across the three quality classes (**Low**, **Medium**, **High**). The matrices show where the models made correct predictions (diagonal elements) and where they misclassified (off-diagonal elements). For example, KNN performed best at distinguishing **Low** and **High** quality classes but struggled with the **Medium** class.



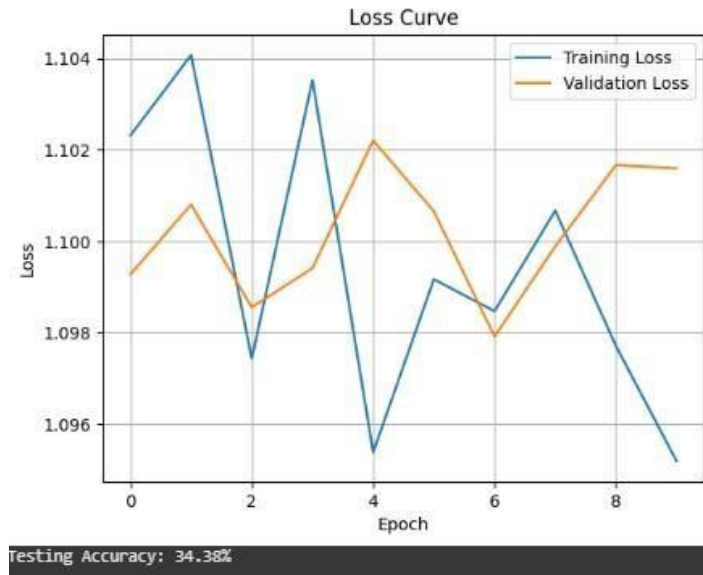
- **ROC Curve:**

The ROC curve comparison illustrates the tradeoff between the True Positive Rate and the False Positive Rate for each model. The KNN model slightly outperformed the others, with the highest AUC for class 1 (Medium). However, all models performed similarly, with marginal differences in AUC values.



- **Loss Curve:**

The loss curve shows the training loss and validation loss over the epochs for the Neural Network model. We observed fluctuations in the training and validation losses, which could indicate overfitting or the need for better tuning of the model's hyperparameters.



7. Conclusion

This project set out to predict software quality by analyzing static code metrics using machine learning techniques. We used a dataset consisting of 1600 instances and 9 features (including Lines of Code, Cyclomatic Complexity, Code Churn, etc.), targeting classification into three quality categories: **Low**, **Medium**, and **High**.

After preprocessing and exploratory data analysis, we trained and evaluated three models:

K-Nearest Neighbors (KNN), **Naive Bayes**, and **Multilayer Perceptron (Neural Network)**. Among these, **KNN achieved the highest accuracy of 38.12%**, followed by Naive Bayes (34.58%) and MLP (34.38%). While KNN outperformed the others marginally, all models showed relatively low overall accuracy and struggled especially with distinguishing the **Medium** class, as revealed in the confusion matrices.

From the results, it is evident that the current feature set has **weak linear correlations** with the target variable. The correlation matrix confirmed this, with most features showing near-zero or negative correlations with **Quality_Label**. For example,

Code_Owner_Experience had a correlation of only 0.03, and **Num_Bugs** only 0.016. Such low predictive power likely explains why even advanced models like MLP failed to significantly outperform simpler ones like Naive Bayes.

Model performance observations:

- **KNN** benefited from normalized feature ranges, but its reliance on distance metrics may have been hindered by feature sparsity and overlap between classes.
- **Naive Bayes** struggled due to its strong assumption of feature independence, which doesn't hold in real-world code metrics.

- **MLPClassifier** exhibited training/validation loss fluctuations, likely due to small dataset size, class overlaps, or suboptimal hyperparameter tuning.

Challenges encountered:

1. **Imbalanced class distribution:** Although not extreme, the classes were uneven (35.4% High, 33.3% Medium, 31.3% Low), which likely impacted classification sensitivity—especially for the Medium class.
2. **Weak feature relevance:** The features used, while meaningful for code analysis, did not show strong discriminative power for quality prediction.
3. **Model tuning limitations:** Time and resource constraints limited our ability to perform deeper hyperparameter tuning, cross-validation, or ensemble modeling, all of which could improve results.
4. **Data noise and outliers:** Boxplots revealed significant outliers in features like `Code_Churn` and `Num_Functions`, which may have distorted model learning.

While the models performed modestly, this project demonstrates the feasibility of using machine learning for automated software quality classification. The experience highlighted the need for **more meaningful features**, perhaps incorporating dynamic code behavior, developer activity metrics, or process maturity indicators. Future work should explore **larger datasets**, apply **ensemble techniques**, and consider **feature selection or engineering** to improve model effectiveness.

Despite limitations, the pipeline we built—from data preprocessing to model evaluation—provides a solid foundation for further improvements and has practical implications for automating software quality assurance in early development stages.